Scaling Matrix Multiplication: From CUDA Kernels to Multi-GPU Tensor Parallelism

Aryan Jain* Fanyi Pu* Ze Hong Maxwell Au*
School of Computer Science and Engineering, Nanyang Technological University
*Equal contribution. Authors listed in alphabetical order.

SC4064 GPU Programming — Course Project Report

Abstract. General Matrix Multiplication (GEMM) is the computational backbone of modern deep learning. This report presents a systematic study across three scales: (i) progressive single-GPU CUDA kernel optimization through seven stages—from naive global memory access to warp-level tiling—benchmarked against cuBLAS on NVIDIA H100 GPUs; (ii) intra-node tensor parallelism on 8×H100 connected via NVLink; and (iii) cross-node tensor parallelism on 16×H100 spanning two nodes over 400 Gb/s InfiniBand. Our best custom kernel reaches 32.8 TFLOPS, 63% of cuBLAS throughput. Strong scaling on 8×H100 for $N = 16384$ yields a 7.03 × speedup (88% efficiency) at 360 TFLOPS aggregate; scaling to 16 GPUs across the IB fabric pushes the $N = 32768$ workload to 667 TFLOPS aggregate. As local GEMM kernels approach peak efficiency, the communication-to-computation ratio rises from 0.22 (naive) to 0.88 (cuBLAS), quantifying the crossover at which inter-GPU communication becomes the dominant bottleneck. A transport sweep confirms that NCCL's IB path is $\sim 125\times$ faster than the TCP fallback at the largest matrix size.

1 Introduction

GEMM operations underpin virtually all compute-intensive workloads in deep learning [1], [2]. In Transformer architectures, multi-head attention and feed-forward layers are fundamentally matrix multiplications. As models scale beyond the memory capacity of a single accelerator, *tensor parallelism* [3] has become indispensable, distributing weight matrices across GPUs at the cost of inter-GPU communication.

While vendor-tuned libraries such as cuBLAS [4] deliver near-optimal single-GPU performance, they abstract away the complex interaction between hardware-level compute intensity and system-level communication latency [5]. This project bridges that gap by:

1. Implementing seven progressively optimized CUDA GEMM kernels (plus one deliberately uncoalesced negative control) to understand hardware-level constraints (memory coalescing, shared memory tiling, register blocking, warp scheduling).
2. Building a distributed tensor-parallel linear layer (forward and backward) using NCCL, including a complete parallel MLP block and a communication–computation overlap variant.
3. Quantifying how local kernel efficiency impacts multi-GPU scalability, identifying the *crossover point* where communication dominates, and extending the study across the node boundary to measure how that crossover shifts when NVLink gives way to InfiniBand.

All experiments are conducted on the hardware described in Section 1.1.

1.1 Experimental Setup

Single-node experiments run on 8 NVIDIA H100 80 GB SXM5 GPUs interconnected via NVLink 4.0 (NV18 topology, 900 GB/s bidirectional per GPU). Multi-node experiments span two such nodes (16 GPUs total) connected by 4×Mellanox ConnectX-7 400 Gb/s InfiniBand HCAs per node with GPUDirect RDMA. Both configurations use CUDA 13.1 and NCCL 2.29.3. The full hardware and software configuration is detailed in Table 6 (Appendix).

2 Background

2.1 GPU Memory Hierarchy

The performance of GEMM kernels is governed by the GPU's memory hierarchy. Table 1 summarizes the key levels for the H100.

Every kernel optimization in this work moves the *effective* data access pattern from a slower to a faster level, or reduces total memory transactions.

2.2 Roofline Model

The *roofline model* [6] characterizes achievable performance as a function of *operational intensity* (OI, FLOP/byte):

$$P_{\max} = \min(\pi, \beta \times \mathrm{OI}) \tag{1}$$

where π is peak FP32 compute throughput and β is peak HBM bandwidth. For H100, counting the fused multiply-

Level	Size	Bandwidth	Latency	Scope
Registers	256 KB/SM	$\sim$ 20 TB/s	0 cycles	Per-thread
Shared Memory	228 KB/SM	$\sim$ 19 TB/s	$\sim$ 20 ns	Per-block
L2 Cache	50 MB	$\sim$ 5 TB/s	$\sim$ 200 ns	Global
HBM3 (Global)	80 GB	$\sim$ 3.35 TB/s	$\sim$ 400 ns	Global

Table 1: H100 GPU memory hierarchy. Each optimization moves data access to a faster level.

add as two FLOPs, $\pi = 132$ SMs $\times$ 128 ALU $\times$ 1.98 GHz $\times$ $2 \approx 67$ TFLOPS and $\beta \approx 3.35$ TB/s, giving a ridge point of $\mathrm{OI} = \pi/\beta \approx 20$ FLOP/byte that separates the memory-bound (left) and compute-bound (right) regimes. For a square GEMM of size N in FP32, the theoretical OI is $2N^3/(3N^2 \cdot 4) = N/6$, which exceeds the ridge for $N \geq 120$—in practice, only the register-blocked kernels realize a meaningful fraction of that ceiling.

2.3 Tensor Parallelism

Tensor parallelism [3] distributes the weight matrices of a linear layer $Y = XW$ across p GPUs. Two complementary partitioning strategies exist.

2.3.1 Column Parallelism

The weight matrix $W \in \mathbb{R}^{K\times N}$ is partitioned column-wise into p shards:

$$W = \left[W_1 \mid W_2 \mid \cdots \mid W_p\right], \quad W_i \in \mathbb{R}^{K\times N/p} \tag{2}$$

The input $X \in \mathbb{R}^{M\times K}$ is replicated on all GPUs. Each GPU i computes a local GEMM:

$$Y_i = XW_i \in \mathbb{R}^{M\times N/p} \tag{3}$$

The full output is assembled via an `AllGather` collective:

$$Y = \left[Y_1 \mid Y_2 \mid \cdots \mid Y_p\right] = XW \in \mathbb{R}^{M\times N} \tag{4}$$

Backward pass. Given upstream gradient $\mathrm{d}Y = \left[\mathrm{d}Y_1 \mid \cdots \mid \mathrm{d}Y_p\right]$, each GPU i computes:

$$\mathrm{d}W_i = X^\top \mathrm{d}Y_i \in \mathbb{R}^{K\times N/p} \quad \text{(local, no communication)} \tag{5}$$

$$\mathrm{d}X_i = \mathrm{d}Y_i W_i^\top \in \mathbb{R}^{M\times K} \quad \text{(partial input gradient)} \tag{6}$$

Since each GPU sees only its column shard, $\mathrm{d}X_i$ captures only the contribution from W_i. The full input gradient requires an `AllReduce` (sum):

$$\mathrm{d}X = \sum_{i=1}^{p} \mathrm{d}X_i = \mathrm{d}YW^\top \tag{7}$$

2.3.2 Row Parallelism

The weight matrix is partitioned row-wise:

$$W = \begin{pmatrix} W_1 \\ W_2 \\ \vdots \\ W_p \end{pmatrix}, \quad W_i \in \mathbb{R}^{K/p\times N} \tag{8}$$

The input $X \in \mathbb{R}^{M\times K}$ is correspondingly split column-wise: $X = \left[X_1 \mid X_2 \mid \cdots \mid X_p\right]$ with $X_i \in \mathbb{R}^{M\times K/p}$. Each GPU i computes:

$$Y_i = X_i W_i \in \mathbb{R}^{M\times N} \quad \text{(partial sum)} \tag{9}$$

The full output is obtained by an `AllReduce`:

$$Y = \sum_{i=1}^{p} Y_i = \sum_{i=1}^{p} X_i W_i = XW \tag{10}$$

Backward pass. Given upstream gradient $\mathrm{d}Y \in \mathbb{R}^{M\times N}$ (replicated after the forward `AllReduce`):

$$\mathrm{d}W_i = X_i^\top \mathrm{d}Y \in \mathbb{R}^{K/p\times N} \quad \text{(local)} \tag{11}$$

$$\mathrm{d}X_i = \mathrm{d}YW_i^\top \in \mathbb{R}^{M\times K/p} \quad \text{(local)} \tag{12}$$

No communication is needed in the backward pass—$\mathrm{d}X_i$ is precisely the gradient for the local input shard.

2.3.3 Composing a Parallel MLP Block

A standard Transformer MLP block computes $\mathrm{MLP}(X) = \sigma(XW_1)W_2$, where σ is an activation function. Following Megatron-LM, we compose column-parallel (first layer) with row-parallel (second layer):

The key insight (Table 2) is that column-parallel's output H_i is *already* the correct input shard for the subsequent row-parallel layer—no `AllGather` is needed between layers. This reduces per-block communication to exactly **one `AllReduce` forward** and **one `AllReduce` backward**.

2.3.4 Communication Cost Model

For a ring-based `AllReduce` with p GPUs, message size S bytes, per-hop bandwidth B, and latency L:

$$T_{\text{AllReduce}} \approx 2\frac{p-1}{p} \cdot \frac{S}{B} + 2(p-1) \cdot L \tag{13}$$

The first term (bandwidth) dominates for large messages. For column-parallel `AllGather`:

$$T_{\text{AllGather}} \approx \frac{p-1}{p} \cdot \frac{S}{B} + (p-1) \cdot L \tag{14}$$

As $p \to \infty$, both approach S/B (bandwidth-limited). This means communication time depends primarily on *data volume* ($S = MN \cdot 4$ bytes for FP32) and interconnect bandwidth, not on the local GEMM kernel speed—a fact we verify experimentally in Section 5.3.

Step	**Operation**	**Per-GPU Compute**	**Communication**
Fwd Layer 1	Column parallel	$H_i = XW_{1,i},\ H_i \in \mathbb{R}^{M\times H/p}$	None
Fwd Activation	Element-wise	$H_i \leftarrow \sigma(H_i)$	None
Fwd Layer 2	Row parallel	$Y_i = H_i W_{2,i},\ Y_i \in \mathbb{R}^{M\times N}$	`AllReduce`(Y)
Bwd Layer 2	Row backward	$\mathrm{d}W_{2,i}, \mathrm{d}H_i$ (local)	None
Bwd Layer 1	Column backward	$\mathrm{d}W_{1,i}$ (local), $\mathrm{d}X_i$ (partial)	`AllReduce`$(\mathrm{d}X)$

Table 2: Communication pattern in a parallel MLP block. Only two `AllReduce` operations are needed: one in the forward pass and one in the backward pass.

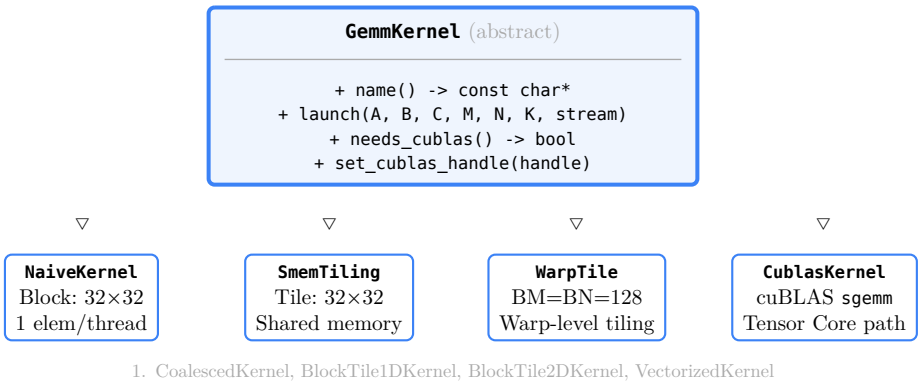

Figure 1: Kernel class hierarchy. Each concrete kernel encapsulates its CUDA `__global__` function, block/grid configuration, and launch logic. The abstract base provides a uniform `launch()` interface.

3 System Design

The codebase is organized into three loosely-coupled layers: a **kernel abstraction layer** with pluggable GEMM implementations, a **CUDA resource layer** with RAII wrappers, and the **tensor parallelism layer** that composes both.

3.1 Kernel Abstraction and Registry

All GEMM kernels—from the naive baseline to cuBLAS—implement a common abstract interface:

Each kernel file self-registers via a static initializer, eliminating the need for centralized dispatch tables:

```
// In naive.cu
class NaiveKernel : public GemmKernel { ... };
namespace { static int reg = KernelRegistry::add(
    std::make_unique<NaiveKernel>()); }
```

The `KernelRegistry` singleton manages all kernel instances. Adding a new kernel requires only creating a single `.cu` file—no header modifications or array synchronization needed. This *open-closed* design replaces the previous approach of parallel enum/function-pointer arrays that required manual synchronization across four separate data structures.

3.2 RAII Resource Management

All GPU resources are managed via move-only RAII wrappers, ensuring deterministic cleanup and preventing leaks:

`DeviceMatrix` composes `CudaMemory<float>` with row/column dimensions, providing `init_random()`, `zero()`, and host-transfer methods. The benchmark code uses `DeviceMatrix` throughout, reducing buffer management from ∼10 lines per matrix to a single constructor call.

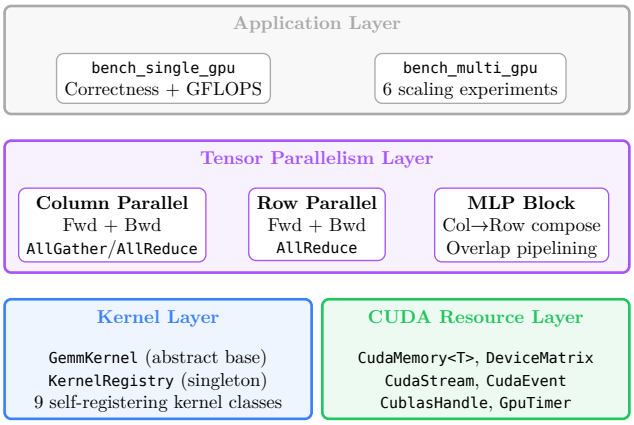

Figure 2: System architecture. The three layers are loosely coupled: kernels are pluggable via the registry, RAII wrappers manage GPU resources, and the tensor parallelism layer composes both.

3.3 Architecture Overview

The tensor parallelism layer (Figure 2, middle) consumes kernels through the `const GemmKernel&` interface, remaining agnostic to the specific kernel implementation. Gradient GEMM operations (transpose + multiply) are factored into reusable `grad_gemm_at_b` and `grad_gemm_a_bt` helpers, reducing the ∼200 lines of duplicated backward-pass code to two composable primitives.

Class	Manages	Acquire	Release
`CudaMemory<T>`	Device allocation	`cudaMalloc`	`cudaFree`
`DeviceMatrix`	2D float matrix	Delegates to `CudaMemory`	Automatic
`CudaStream`	CUDA stream	`cudaStreamCreate`	`cudaStreamDestroy`
`CudaEvent`	CUDA event	`cudaEventCreate`	`cudaEventDestroy`
`CublasHandle`	cuBLAS context	`cublasCreate`	`cublasDestroy`

Table 3: RAII resource wrappers. All classes delete copy operations and implement move semantics, preventing accidental double-free or resource leaks.

Kernel	Technique	Key Idea	Effect
1. Naive	Baseline	1 thread → 1 element, K global loads/FMA	$\mathcal{O}(1)$ FLOP/byte
2. Coalesced	Memory coalescing	threadIdx.x → column for stride-1	Fewer transactions
3. Shared Mem	SRAM tiling	32×32 tiles reused $32\times$	$32\times$ less traffic
4. 1D BlockTile	Thread coarsening	Each thread → TM=8 rows	Better smem reuse
5. 2D BlockTile	Register blocking	TM×TN= 8×8 per thread	$8\times$ fewer smem reads
6. Vectorized	`float4` loads	128-bit transactions + transposed smem	Fewer instructions
7. WarpTile	Warp-level tiling	Block → Warp → Thread hierarchy	Better L1 locality

Table 4: Progressive GEMM kernel optimization roadmap.

4 Single-GPU Kernel Optimization

4.1 Optimization Roadmap

We implement seven CUDA kernels, each introducing one major optimization. Table 4 summarizes the progression.

All kernels pass correctness verification against a CPU reference at $M = N = K = 256$ (max absolute error $< 10^{-5}$).

4.2 Results and Analysis

Figure 3 shows the performance progression. Key observations:

- **Naive and Coalesced kernels** plateau at $\sim 6\{,\}300$ GFLOPS regardless of matrix size, confirming they are memory-bandwidth-bound. The coalesced variant is $12.7\times$ faster than the deliberately uncoalesced control (6,338 vs. 498 GFLOPS at $N = 4096$), a vivid demonstration that uncoalesced global-memory accesses alone can squander more than an order of magnitude of throughput.
- **Shared memory tiling** (Kernel 3) raises throughput to $\sim 9\{,\}000$ GFLOPS by reducing global traffic by $32\times$, but remains below the compute-bound regime.
- **Register blocking** (Kernels 4–7) produces the most dramatic gains. The 2D block tile reaches 22,062 GFLOPS at $N = 4096$—a $2.4\times$ improvement over shared memory alone. Vectorized `float4` loads push this to 32,492 GFLOPS at $N = 4096$, and the warp-tiled variant sustains 32,820 GFLOPS at $N = 16384$.
- **cuBLAS** peaks at 51,662 GFLOPS at $N = 8192$ and sustains ~ 51 TFLOPS thereafter, about 77% of the 67 TFLOPS FP32 ceiling. It outperforms our best hand-written kernel by $1.59\times$; the gap is attributable to tensor-core paths (TF32 with FP32 accumulation) and extensive offline autotuning that an FP32-only implementation cannot replicate.

Figure 4 shows the relative trajectory. For $N \geq 2048$, the gap to cuBLAS narrows monotonically with optimization level—11% (naive) → 18% (shared memory) → 43% (2D block) → 63% (vectorized/warp). The crossover behaviour

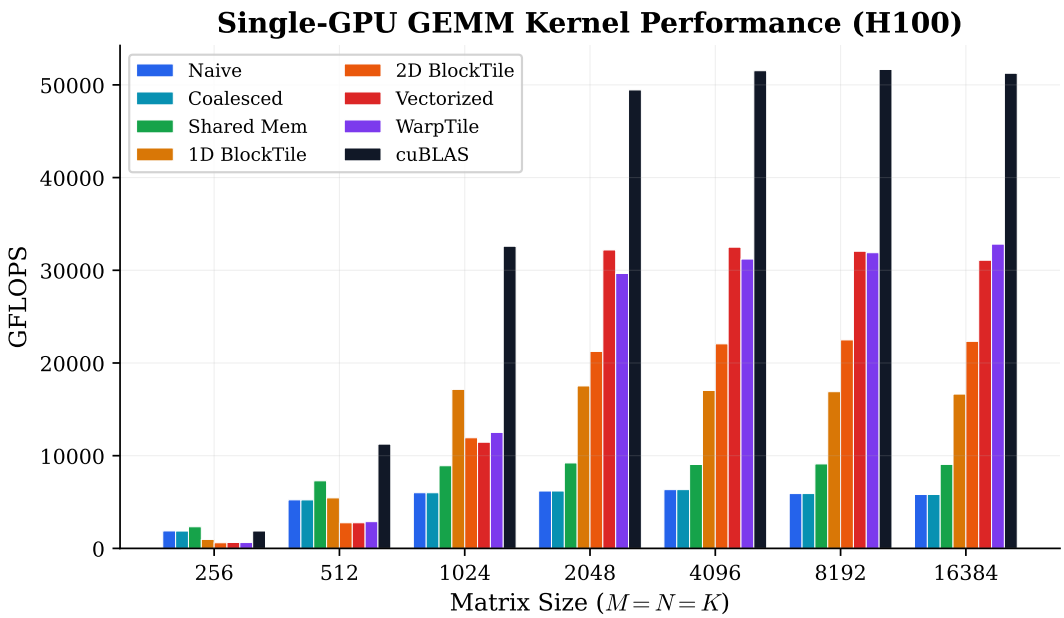

Figure 3: GFLOPS across all kernel optimization stages and matrix sizes (the Uncoalesced negative control is omitted; see discussion). Register-blocked kernels (2D BlockTile, Vectorized, WarpTile) climb sharply with size as the working set outgrows the overhead of thread setup; cuBLAS sits on a separate tier courtesy of its tensor-core path.

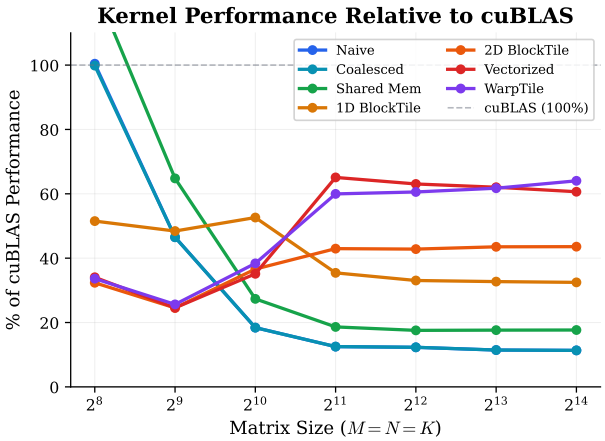

Figure 4: Each kernel's throughput as a percentage of cuBLAS. The Vectorized kernel reaches 63% of cuBLAS at $N = 4096$; at very small sizes ($N = 256$) the Shared-Memory kernel briefly matches cuBLAS because cuBLAS's dispatch overhead dominates its measured time.

below $N = 1024$ reflects small-size launch and library-dispatch overheads rather than steady-state throughput.

The roofline view (Figure 5) reinforces the takeaway: at sizes relevant for deep learning, SGEMM is not memory-bound in principle—the ridge point lies two orders of magnitude to the left of the workload's OI. The achieved performance therefore measures how close each kernel comes to the flat compute roof. cuBLAS reaches 77% of peak; the warp-tiled kernel, 49%; the naive kernel, 9.4%. The missing compute ceiling for hand-written kernels is not algorithmic but organizational: register-file pressure, scheduling of FMAs inside a warp, and tensor-core utilization all separate a respectable kernel from a peak one.

5 Intra-Node Tensor Parallelism (NVLink)

We implement the tensor-parallel primitives described in Table 2 using NCCL. Each GPU runs in a separate host thread with its own CUDA stream and cuBLAS handle; NCCL communicators for each target GPU count are pre-initialised via `ncclCommInitAll` at startup. The implementation includes:

- **Column-parallel forward/backward**: local GEMM + `ncclAllGather` (forward) / `ncclAllReduce` (backward).
- **Row-parallel forward/backward**: local GEMM + `ncclAllReduce` (forward) / no communication (backward).
- **Parallel MLP block**: composed column → row parallelism, matching the two-`AllReduce` pattern in Table 2.
- **Communication–computation overlap**: the output matrix is chunked along the M dimension; each chunk's `AllReduce` is pipelined with the next chunk's GEMM on a separate CUDA stream, synchronised via `cudaEvent`.

All intra-node experiments use 8×H100 GPUs connected via NVLink 4.0. Backward passes that call our row-major custom kernels use `cublasSgeam` for explicit transposition, since those kernels lack a built-in transposed variant.

5.1 Strong Scaling

Figure 6 summarises the scaling behaviour. The $N = 16384$ workload scales from 171.6 ms (1 GPU) to 24.4 ms (8 GPUs), a 7.03 × speedup and **87.8% parallel efficiency**, reaching 360 TFLOPS aggregate throughput. At $N = 8192$, the curve flattens earlier (5.8 × at 8 GPUs) because the 128 MB `AllGather` payload already costs a non-trivial fraction of the shorter GEMM. The $N = 2048$ line is the cautionary tale: past 4 GPUs the 16 MB `AllGather` takes longer than the shard-sized GEMM, so adding GPUs **increases** wall-clock time. The jog at 8→16 GPUs for $N = 2048$ and $N = 4096$ marks the transition from NVLink to cross-node IB (see Section 7).

Figure 7 makes the same story explicit. Large matrices retain high efficiency because compute grows as $O(N^3)$ while communication volume grows only as $O(N^2)$. Small matrices dip below 10% efficiency within the NVLink domain.

5.2 Weak Scaling

Under weak scaling (Figure 8), the 2048 tile incurs the largest relative slowdown (2.75 × at 8 GPUs) because

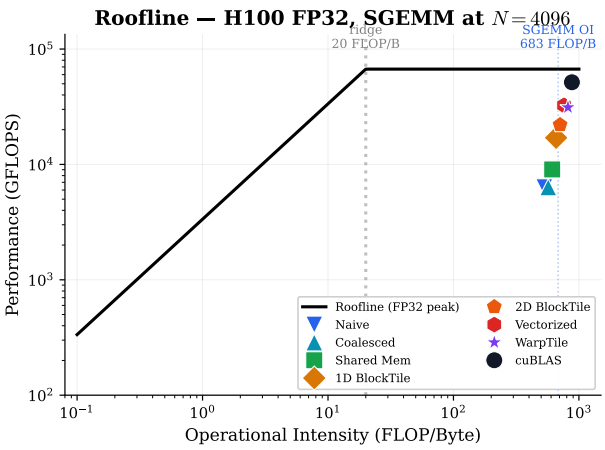

Figure 5: Roofline analysis at $N = 4096$. The FP32 ceiling is 67 TFLOPS (FMA-counted) and the ridge point is at 20 FLOP/byte; SGEMM at $N = 4096$ has an operational intensity of 683 FLOP/byte, deeply in the compute-bound regime. All kernels are plotted at that OI with slight horizontal jitter; vertical position therefore measures their achieved fraction of peak compute.

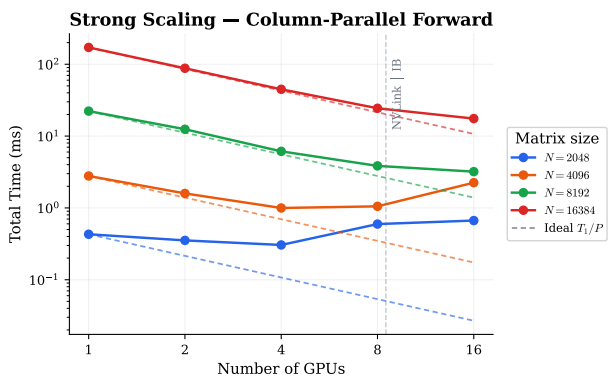

Figure 6: Strong scaling: total wall-clock time vs. number of GPUs for four matrix sizes, combining single-node (1, 2, 4, 8 GPUs over NVLink) and cross-node (16 GPUs over IB) measurements on a single axis. Dashed lines give ideal T_1/P scaling. Larger matrices scale substantially further before communication dominates.

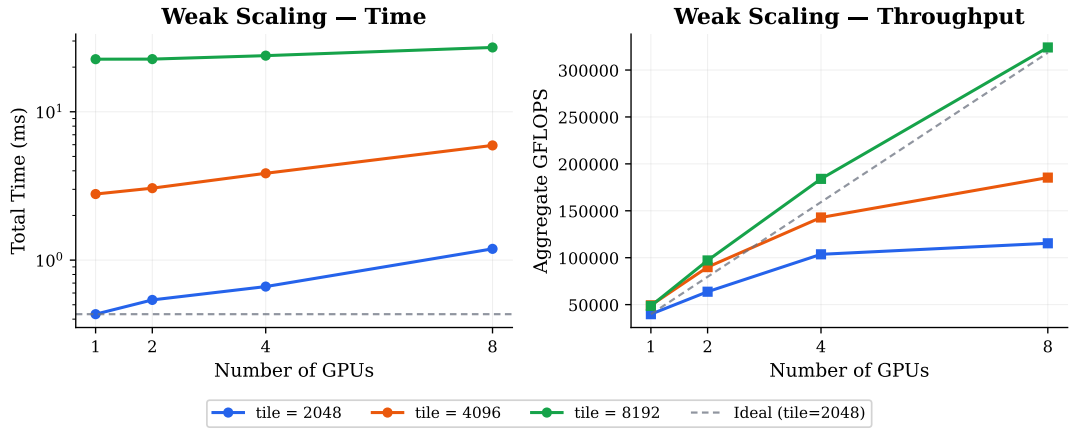

Figure 8: Weak scaling: three per-GPU tile sizes (2048, 4096, 8192) held fixed while the number of GPUs increases. Left: total time (log scale) grows mildly—the overhead is entirely from `AllGather`. Right: aggregate throughput in GFLOPS approaches linear scaling as the per-GPU tile grows.

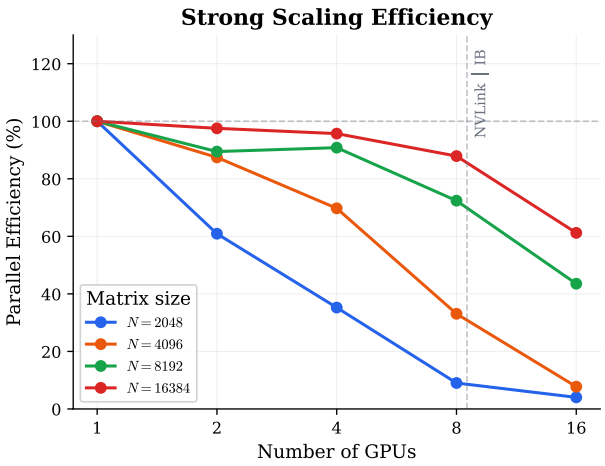

Figure 7: Parallel efficiency $\eta = T_1/(P \cdot T_P)$. At $N = 16384$, efficiency remains 88% at 8 GPUs on NVLink and 61% at 16 GPUs across two nodes; small matrices collapse to near-zero efficiency well before the node boundary.

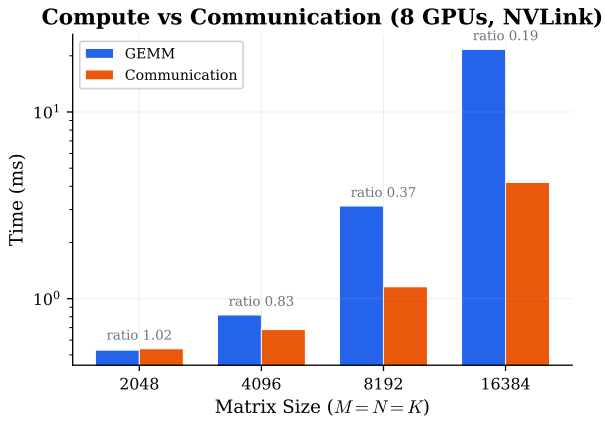

Figure 9: GEMM time vs. communication time at different matrix sizes on 8 GPUs (cuBLAS kernel, NVLink). The annotated ratio drops from 1.02 at $N = 2048$ to 0.19 at $N = 16384$ as compute grows $O(N^3)$ while the `AllGather` payload grows only $O(N^2)$.

`AllGather` latency dominates; the 8192 tile stays within 1.20 × of the single-GPU time and reaches 324 TFLOPS aggregate, 6.7 × the single-GPU baseline at that tile size. Weak scaling therefore acts as a direct probe of the communication-to-computation ratio at each tile size.

5.3 Communication–Computation Analysis

This is the central experiment of the project: *how does the communication-to-computation ratio evolve as local kernels are optimised, and how does it scale with matrix size?*

Figure 9 captures the cubic-vs-quadratic scaling law. At $N = 2048$, communication (0.54 ms) and compute (0.53 ms) are essentially equal. By $N = 16384$, compute grows to 21.7 ms while communication is only 4.2 ms—a ratio of 0.19. This matches the asymptotic expectation $\sim N/B$ for ratio(N) and predicts that contemporary Transformer hidden dimensions ($N \geq 8192$, as in Llama-2 70B or GPT-3 175B) sit comfortably in the compute-dominated regime.

Figure 10 is the central result of the project. Communication time is the same (≈ 0.69 ms) for every local kernel—it is a property of the `AllGather` payload and the NVLink fabric, not of the local GEMM. As the local GEMM accelerates from 3.20 ms (Naive) to 0.78 ms (cuBLAS), the ratio rises from 0.22 to 0.88. Interpreting the extremes:

- With a **naive** kernel, communication is only 22% of compute: further kernel optimisation translates almost directly into end-to-end speedup.
- With **cuBLAS**, communication has already climbed to 88% of compute: even infinite further GEMM speedup would buy at most a 1.88 × end-to-end improvement. The system is within a factor of two of the *communication-bound* regime.

The crossover is the fundamental tension of distributed deep learning: once local compute is fast enough, only the interconnect matters.

5.4 MLP Forward and Backward

Figure 11 shows the MLP block's forward and backward time. At $N \geq 8192$, the backward pass is almost exactly 2 × the forward, consistent with the algorithmic count (2 GEMMs forward vs. 4 GEMMs backward, each pass sharing a single `AllReduce`). At small sizes, the ratio inflates (10.3 × at $N = 2048$) because fixed per-call launch and synchronisation overhead in the backward's extra kernels dwarfs the actual arithmetic. This is an artefact of the

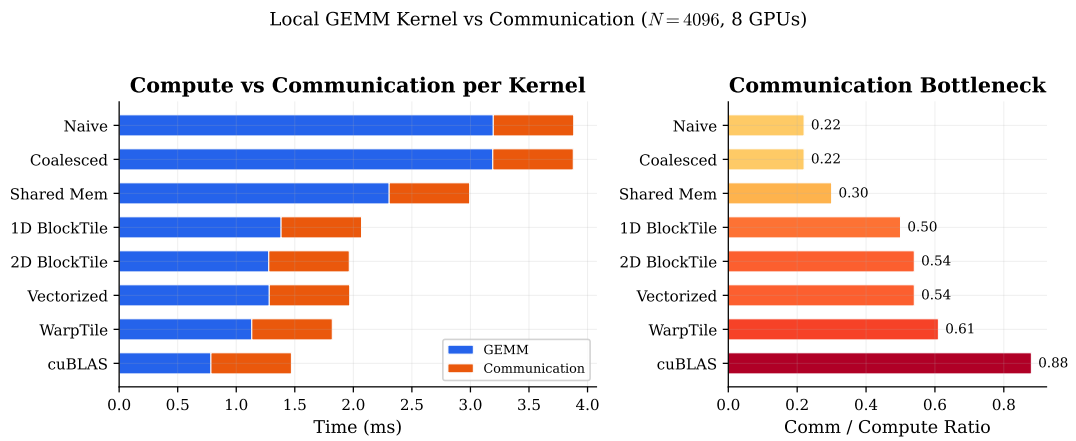

Figure 10: Left: stacked absolute GEMM and communication time per kernel at $N = 4096$, 8 GPUs (the Uncoalesced kernel is dropped because its 35 ms GEMM compresses the chart). Right: communication-to-computation ratio rising monotonically from 0.22 (Naive) to 0.88 (cuBLAS) as the local GEMM gets faster.

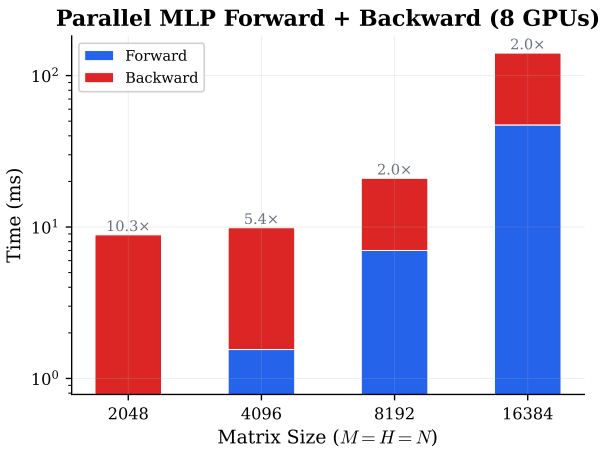

Figure 11: Parallel MLP (column → row) forward + backward on 8 GPUs. As the matrix grows, the backward/forward ratio converges to its algorithmic limit of 2 × (2 GEMMs forward, 4 GEMMs backward).

launch-overhead regime rather than a property of the algorithm.

5.5 Communication–Computation Overlap

The overlap experiment (Figure 12) splits the output along the M dimension into four chunks, pipelining each chunk's `AllReduce` with the next chunk's GEMM on a separate CUDA stream. At $N = 2048$ and $N = 4096$, overlap actually hurts (0.90 × and 0.94 ×): the per-chunk GEMM is too small to hide the extra event synchronisation. At $N = 8192$ the nominal speedup is 1.25 ×, but the no-overlap baseline has 73% coefficient of variation in that row (`Total_std` of 3.8 ms on 5.2 ms mean), so the measurement is dominated by outliers. The most reliable point is $N = 16384$, where overlap delivers a clean 1.06 × with low variance. On H100 with NVLink, `AllReduce` latency is only ∼ 3 ms even for 1 GB payloads, so the hideable window is small. We show in Section 7 that overlap becomes considerably more valuable when the fabric latency rises.

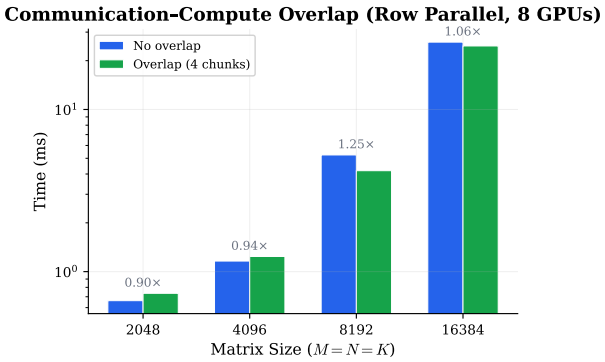

Figure 12: Row-parallel forward with and without chunked overlap (4 chunks). Overlap is neutral-to-harmful at small sizes (event-sync overhead dominates), helps at $N = 8192$ (1.25 × nominal, high variance), and settles to a modest 1.06 × at $N = 16384$.

6 Cross-Node Tensor Parallelism (InfiniBand)

To characterise tensor parallelism beyond the NVLink domain, we replicate the column-parallel strong-scaling and ratio-vs-size experiments on 16 GPUs spanning two nodes connected by 4×400 Gb/s InfiniBand. NCCL is forced into each of four transport configurations in a single benchmark run so that all transports see the same process layout and GPU placement:

All four runs verify that the `auto`, `ib`, and `ring` paths use `NET/IB` with `GDRDMA` over all four HCAs (`mlx5_0/1/4/5`), while `tcp` falls back to plain sockets as intended.

6.1 Transport Sweep: IB vs. TCP

Figure 13 is the headline result for the multi-node study. At $N = 32768$, the IB-backed transports move the 4 GB `AllGather` payload in 23.3–25.3 ms (consistent with ∼ 175 GB/s effective goodput per GPU), while the TCP fallback requires 2.93 *seconds*—a 125 × slowdown that completely dominates every other cost in the system. The right panel translates this into ratio-vs-size: the IB ratio peaks at 1.4 around $N = 4096$ and then decreases as compute takes over, whereas TCP peaks at over 100 × the compute time at $N = 8192$ and remains in the communication-bound regime across the entire measured range.

The practical implication is stark: a hardware-adequate cluster (IB 400 Gb/s with GDRDMA) and a software fallback (TCP over ethernet) are not a spectrum of per-

Tag	Environment
`auto`	NCCL default selection (picks IB + GDRDMA on this system)
`ib`	`NCCL_NET=IB NCCL_IB_DISABLE=0`
`ring`	`NCCL_ALGO=Ring`, otherwise auto
`tcp`	`NCCL_IB_DISABLE=1` — forces the TCP socket plugin

Table 5: Transport configurations swept across the 16-GPU run. `NCCL_ALGO=Tree` was attempted but NCCL does not support `AllGather` under the Tree algorithm, so that variant is omitted.

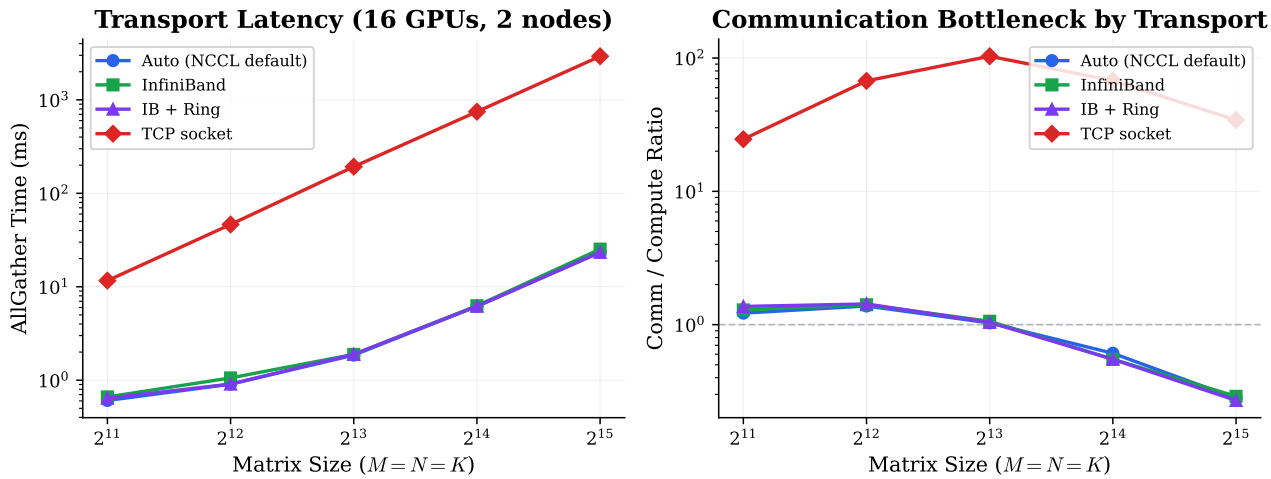

Figure 13: Left: per-size `AllGather` time at 16 GPUs across the four transport configurations (log-log). Right: the resulting communication/computation ratio for the same run. IB, auto, and ring trace the same curve; TCP lies two orders of magnitude above.

formance, they are different regimes of operation. In the TCP case, accelerating the local GEMM by any amount is essentially invisible.

6.2 Strong Scaling Across the Node Boundary

The right-most data points in Figure 6 and Figure 7 are taken from this run. Three observations:

1. **The node boundary is visible but not catastrophic for compute-heavy shapes.** $N = 16384$ goes from 24.4 ms at 8 NVLink GPUs to 17.5 ms at 16 IB GPUs—a further 1.39 × speedup. The efficiency drop from 88% to 61% is the cost of crossing the IB fabric.
2. **Very large matrices cross the boundary cheaply.** The $N = 32768$ workload on 16 GPUs hits 667 TFLOPS aggregate (compute 86 ms, communication 23 ms), the largest throughput we measure anywhere. Its communication/computation ratio at 16 GPUs is 0.28, still well inside the compute-dominated regime.
3. **Small matrices are punished by the fabric jump.** $N = 2048$ and $N = 4096$ go *slower* when moving from NVLink 8-GPU to IB 16-GPU because both the `AllGather` latency and the shrunken local GEMM move in the wrong direction.

Taken together with the intra-node ratio experiment, the multi-node data gives a concrete prescription: choose tensor parallelism width such that the sharded GEMM stays comfortably to the left of the bandwidth-limited `AllGather`. On this cluster, that threshold is $N \approx 8192$ for 8-way NVLink and $N \approx 16384$ for 16-way across IB.

7 Conclusion

This project provides a comprehensive study of GEMM optimisation and tensor parallelism at three scales—a single GPU, a single-node multi-GPU domain over NVLink, and a cross-node deployment over InfiniBand.

Single-GPU kernel optimisation. Seven progressively optimised CUDA kernels trace the path from memory-bound (∼ 6.3 TFLOPS) to near-compute-bound (∼ 32.8 TFLOPS) FP32 SGEMM. The dominant improvement is 2D register blocking, which amortises shared-memory reads across many FMAs. Our best hand-written kernel reaches 63% of cuBLAS, and the remaining gap is attributable almost entirely to cuBLAS's TF32 tensor-core path and offline autotuning.

Intra-node tensor parallelism. On 8×H100 NVLink, strong scaling at $N = 16384$ delivers a 7.03 × speedup (88% efficiency) and 360 TFLOPS aggregate throughput. The communication-to-computation ratio rises monotonically from 0.22 (naive) to 0.88 (cuBLAS) as the local GEMM gets faster: a direct measurement of the crossover where system-level communication begins to dominate.

Cross-node tensor parallelism. Extending to 16 GPUs over 400 Gb/s IB, the $N = 32768$ workload reaches 667 TFLOPS aggregate. A transport sweep quantifies how much the fabric matters: forcing NCCL onto plain TCP sockets inflates the 4 GB `AllGather` by 125 ×, pushing the ratio to > 100 and swamping any kernel-level optimisation.

Software engineering. The codebase follows an object-oriented, RAII-based design with a self-registering

Component	Specification
GPU (per node)	8 × NVIDIA H100 80 GB HBM3 (SXM5)
Intra-node interconnect	NVLink 4.0, all-to-all NV18 (900 GB/s bidirectional per GPU)
Inter-node interconnect	4 × Mellanox ConnectX-7 400 Gb/s InfiniBand per node, GPUDirect RDMA
GPU Compute	132 SMs, 1980 MHz boost, sm_90, 67 TFLOPS FP32 dense peak (FMA-counted)
GPU Memory	80 GB HBM3, ∼3.35 TB/s bandwidth per GPU
CPU (per node)	2 × Intel Xeon Gold 6448Y (32 cores / 64 threads each, 128 threads total)
System Memory (per node)	2 TB DDR5
OS	Ubuntu 24.04.4 LTS (kernel 5.14.0)
CUDA Toolkit	13.1
NCCL	2.29.3
GPU Driver	550.90.07
Max world size measured	16 GPUs (2 nodes × 8 GPUs)

Table 6: Hardware and software configuration. Single-node runs use one node with 8 GPUs over NVLink; multi-node runs span two nodes over 400 Gb/s InfiniBand.

kernel registry, move-only resource wrappers (no manual `cudaFree`), and composable gradient-GEMM helpers. New kernels are added through a single file with zero changes to any existing code.

Taken together, the results support a simple design rule: at any given hardware scale, choose tensor-parallel width so the sharded local GEMM stays to the left of the interconnect-bound `AllGather`/`AllReduce`—and verify that the interconnect is actually doing what you think it is.

Bibliography

[1] A. Vaswani *et al.*, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[2] Z. Jia, M. Maggioni, B. Staiger, and D. P. Scarpazza, "Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking," *arXiv preprint arXiv:1804.06826*, 2018.

[3] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," *arXiv preprint arXiv:1909.08053*, 2019.

[4] NVIDIA Corporation, "cuBLAS Library." 2026. [Online]. Available: https://docs.nvidia.com/cuda/pdf/CUBLAS_Library.pdf

[5] NVIDIA Corporation, "NCCL Developer Guide." 2026. [Online]. Available: https://docs.nvidia.com/deeplearning/nccl/user-guide/

[6] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

8 Appendix